LAPORAN PRATIKUM STRUKTUR DATA

LAPORAN PRAKTIKUM PEKAN 3

Disusun Oleh:
Nama: Muhammad Aufa Rafiki

NIM: 2511531012

Dosen Pengampu: Dr. Wahyudi S.T.M.T

Asisten Praktikum: M. Galid

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2025

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya laporan praktikum ini dapat diselesaikan dengan baik. Laporan ini disusun sebagai salah satu bentuk dokumentasi dan pemahaman terhadap materi praktikum struktur data pada pekan ke-3 yang membahas tentang implementasi Stack dalam pemrograman Java.

Dalam praktikum ini, penulis mempelajari berbagai implementasi stack, baik menggunakan library bawaan Java maupun secara manual menggunakan array dan ArrayList. Selain itu, praktikum juga mencakup penerapan stack dalam penyelesaian masalah, seperti pencarian nilai maksimum dan evaluasi ekspresi postfix.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun untuk perbaikan di masa mendatang.

DAFTAR ISI

KATA PENGANTAR i
DAFTAR ISI ii
BAB I PENDAHULUAN 1
1.1 Dasar Teori 1
1.2 Tujuan Pratikum 1
1.3 Manfaat Pratikum 1
BAB II PEMBAHASAN 2
2.1 latihanStack_2511531012 2
2.2 NilaiMaksimum_2511531012 3
2.3 SiswaStack_2511531012 4
2.4 stackArray_2511531012 6
2.5 stackArrayDriver_2511531012 8
2.6 StackPostfix_2511531012 9
BAB III KESIMPULAN 10
DAFTAR PUSTAKA 11
Tugas 12

BAB I
PENDAHULUAN

1.1 Dasar Teori

Stack adalah struktur data linear yang menggunakan prinsip Last In First Out (LIFO), yaitu data yang terakhir masuk akan keluar pertama. Operasi dasar pada stack meliputi push (menambah data), pop (menghapus data teratas), peek (melihat data teratas), dan isEmpty (mengecek apakah stack kosong). Stack dapat diimplementasikan menggunakan array, ArrayList, atau class bawaan Java. Dalam pemrograman, stack banyak digunakan untuk berbagai keperluan seperti evaluasi ekspresi matematika (postfix), rekursi, serta fitur undo dan redo. Dengan memahami stack, mahasiswa dapat meningkatkan logika dalam pengolahan data secara efisien.

1.2 Tujuan Pratikum

Tujuan dari pelaksanaan praktikum antara lain sebagai berikut:

1. Memahami konsep dasar struktur data stack
2. Mempelajari operasi dasar stack (push, pop, peek)
3. Mengimplementasikan stack menggunakan berbagai metode
4. Memahami cara kerja stack dalam pemrograman Java
5. Menerapkan stack dalam penyelesaian masalah seperti pencarian nilai maksimum dan evaluasi postfix

1.3 Manfaat Praktikum

1. Mahasiswa dapat memahami konsep stack secara teori dan praktik
2. Mahasiswa mampu mengimplementasikan stack dalam berbagai bentuk
3. Melatih logika pemrograman dalam pengolahan data
4. Memahami penerapan stack dalam kasus nyata
5. Menjadi dasar untuk materi struktur data yang lebih kompleks

BAB II

PEMBAHASAN

2.1 latihanStack_2511531012

```java
package pekan3_2511531012;
import java.util.Stack;
public class latihanStack_2511531012 {

    public static void main(String[] args) {
        Stack<Integer> s = new Stack<Integer>();
        s.push(42);
        s.push(-3);
        s.push(17);
        System.out.println("nilai stack= "+s);
        System.out.println("nilai pop ="+s.pop());
        System.out.println("nilai stack setelah pop="+s);
        System.out.println("nilai peek"+s.peek());
        System.out.println("nilai stack setelah peek= "+s);


    }

}
```

Penjelasan Program:

1. import java.util.Stack; →Digunakan untuk mengakses class Stack dari library Java.
2. Stack<Integer> s = new Stack<Integer>();
   →Membuat stack bernama s.
   →Tipe data yang disimpan adalah Integer (bilangan bulat).
3. Method push() digunakan untuk menambahkan elemen ke stack.
4. System.out.println("nilai stack= "+s);→Menampilkan seluruh isi stack: [42, -3, 17]
5. s.pop() →Method pop() berfungsi:
   - Menghapus elemen teratas
   - Mengembalikan nilai elemen tersebut
6. Menampilkan Stack Setelah pop →[42, -3]
7. • s.peek() → Method peek() berfungsi: Melihat elemen paling atas tanpa menghapusnya
8. Menampilkan Stack Setelah peek →[42, -3]

2.2 NilaiMaksimum_2511531012

```java
package pekan3_2511531012;
import java.util.Stack;
public class NilaiMaksimum_2511531012 {
    public static  int max (Stack<Integer> s) {
        Stack<Integer> backup= new Stack<Integer>();
            int maxValue = s.pop();
            backup.push(maxValue);
            while (!s.isEmpty()) {
                int next = s.pop();
                backup.push(next);
                maxValue = Math.max(maxValue, next);
            }
            while (!backup.isEmpty()) {
                s.push(backup.pop());
            }
            return maxValue;
        }

    public static void main(String[] args) {
        Stack<Integer> s = new Stack<Integer>();
        s.push(70);
        s.push(12);
        s.push(20);
        System.out.println("isi stack "+s);
        System.out.println("Stack Teratas "+s.peek());
        System.out.println("Nilai maksimum "+max(s));

    }

}
```

Penjelasan Program:

1. Method max(Stack<Integer> s) → Method ini berfungsi untuk mencari nilai maksimum dalam stack.
2. Stack<Integer> backup = new Stack<Integer>(); → Digunakan untuk menyimpan sementara isi stack agar tidak hilang.
3. int maxValue = s.pop(); → Mengambil elemen teratas sebagai nilai awal maksimum.
4. Elemen ini disimpan ke backup → backup.push(maxValue);
5. Perulangan Mencari Nilai Maksimum;
   - while (!s.isEmpty())
   - Selama stack masih ada isinya:
   - Ambil elemen dengan pop()
   - Simpan ke backup
   - Bandingkan dengan nilai maksimum: →maxValue = Math.max(maxValue, next);
6. Mengembalikan Isi Stack:
   - while (!backup.isEmpty())
   - Semua data di backup dikembalikan ke stack awal: → s.push(backup.pop());
   - Agar stack kembali seperti semula (tidak berubah).
7. return maxValue; →Menghasilkan nilai terbesar dalam stack.
8. Method main
9. Membuat stack kosong bernama s. Dan Stack hanya menyimpan data bertipe Integer.
10. Menambahkan Elemen ke Stack:→ s.push(70); s.push(12); s.push(20);
11. Menampilkan seluruh isi stack. →isi stack [70, 12, 20]
12. Menampilkan Elemen Teratas → peek() digunakan untuk melihat elemen paling atas tanpa

menghapusnya.

13. Mencari Nilai Maksimum →Memanggil method max(s) untuk mencari nilai terbesar dalam stack.
    Method ini:
    - Membandingkan semua elemen dalam stack
    - Tidak mengubah isi stack
14. Output: Nilai maksimum 70

2.3 SiswaStack_2511531012

```java
package pekan3_2511531012;
import java.util.ArrayList;

class Siswa_2511531012 {
    String nama;
    int nim;
    public Siswa_2511531012(String nama, int nim) {
        this.nama = nama;
        this.nim = nim;
    }
    public String toString() {
        return "Nim: "+nim + ",Nama: "+ nama;
    }
}
public class SiswaStack_2511531012 {
    private ArrayList<Siswa_2511531012> stack;

    public SiswaStack_2511531012() {
        stack = new ArrayList<>();
    }
    public void push_2511531012(Siswa_2511531012 mhs) {
        stack.add(mhs);
    }
    public Siswa_2511531012 pop() {
        if (!isEmpty()) {
            return stack.remove(stack.size() - 1);
        }
        return null;
    }
    public Siswa_2511531012 peek() {
        if(!isEmpty()) {
            return stack.get(stack.size() - 1);
        }
        return null;
    }
```

```java
    public boolean isEmpty() {
        return stack.isEmpty();
    }
    public void tampilkanSiswa_2511531012() {
        for (int i= stack.size()-1; i>=0; i--) {
            System.out.println(stack.get(i));
        }
    }
    public static void main(String[] args) {
        SiswaStack_2511531012 studentStack = new SiswaStack_2511531012();

        Siswa_2511531012 mhs1 = new Siswa_2511531012("Ali", 1);
        Siswa_2511531012 mhs2 = new Siswa_2511531012("Boby", 2);
        Siswa_2511531012 mhs3 = new Siswa_2511531012("Charles", 3);

        studentStack.push_2511531012(mhs1);
        studentStack.push_2511531012(mhs2);
        studentStack.push_2511531012(mhs3);

        System.out.println("Siswa di dalam stack:");
        studentStack.tampilkanSiswa_2511531012();

        System.out.println("Siswa teratas "+ studentStack.peek());
        System.out.println("mengeluarkan siswa teratas dari stack: "+ studentStack.pop());
        System.out.println("daftar siswa setekah di pop :");
        studentStack.tampilkanSiswa_2511531012();


    }

}
```

Penjelasan Program:

1. Class Siswa_2511531012;
2. Atribut:
   String nama → menyimpan nama siswa
   int nim → menyimpan nomor induk mahasiswa
3. Constructor:
   public Siswa_2511531012(String nama, int nim) → Digunakan untuk mengisi nilai awal objek saat dibuat.
4. Method toString() → Mengubah objek menjadi teks saat ditampilkan.
5. public class SiswaStack_2511531012
6. Atribut:
   ArrayList<Siswa_2511531012> stack →Digunakan sebagai wadah penyimpanan stack.
7. Constructor:
   stack = new ArrayList<>(); → Menginisialisasi stack kosong.
8. push_2511531012()
   Fungsi: menambahkan data ke stack
   Cara kerja: → stack.add(mhs)
   Data ditambahkan ke posisi terakhir (top stack)
9. pop()
   Fungsi: mengambil & menghapus elemen teratas
   Cara kerja: → Menghapus elemen di index terakhir:
   stack.remove(stack.size() - 1)
   Jika kosong → return null
10. peek()
    Fungsi: melihat elemen teratas tanpa menghapus
    Cara kerja:
    stack.get(stack.size() - 1)
    Jika kosong → return null
11. isEmpty()
    Fungsi: mengecek apakah stack kosong

Return:
true → kosong
false → ada isi

12. tampilkanSiswa_2511531012()
    Fungsi: menampilkan isi stack
    Ditampilkan dari atas ke bawah: →Loop dari index terakhir ke awal
13. Method main
14. Membuat Objek Stack
    SiswaStack_2511531012 studentStack
    Stack kosong siap digunakan
15. Membuat Data Siswa
16. Menambahkan ke Stack (push)
    Urutan masuk: Ali, Boby, Charles
17. Menampilkan Isi Stack
18. Melihat Elemen Teratas (peek) → Charles
19. Menghapus Elemen Teratas (pop) → Charles
20. Menampilkan Stack Setelah pop →[Ali, Boby]

2.4 stackArray_2511531012

```java
package pekan3_2511531012;

public class stackArray_2511531012 {
    static final int MAX = 1000;
    int top;
    int a[]= new int[MAX];
    boolean isEmpty_2511531012()
    {
        return (top<0);
    }
    stackArray_2511531012()
    {
        top=-1;
    }
    boolean push_2511531012(int x)
    {
        if(top >=(MAX - 1)) {
            System.out.println("Stack Oveflow");
            return false;
        }
        else {
            a[++top]=x;
            System.out.println(x +" dimasukkan dalam satck");
            return true;
        }
    }
    int pop_2511531012()
    {
        if(top<0) {
            System.out.println("Stack Underflow");
            return 0;
        }
        else {
            int x = a[top--];
            return x;
```

```java
        }

    }
    int peek_2511531012()
    {
        if(top<0) {
            System.out.println("Stack Underflow");
            return 0;
        }
        else {
            int x = a[top];
            return x;
        }
    }
    void print_2511531012() {
        for(int i=top;i>-1;i--) {
            System.out.print(" "+ a[i]);
        }
    }

}
```

Penjelasan Program:

1. static final int MAX = 1000; → Menentukan kapasitas maksimum stack.
2. int top; → Menunjukkan posisi elemen teratas (top stack).
3. int a[] = new int[MAX]; →Array untuk menyimpan data stack.
4. stackArray_2511531012()
   - Mengatur nilai awal: → top = -1
   - Artinya: Stack masih kosong.
5. Method isEmpty_2511531012() → mengecek apakah stack kosong.
   - Kondisi:
     - Jika top < 0 → stack kosong
   - Return:
     - true → kosong
     - false → tidak kosong
6. Method push_2511531012(int x) → menambahkan elemen ke stack.
   Cara kerja:
   - Cek apakah stack penuh:
     - top >= MAX - 1
   - Jika penuh:
     - Menampilkan "Stack Overflow"
   - Jika tidak:
     - a[++top] = x
     - Menambahkan elemen ke posisi atas

   Menampilkan pesan → "x dimasukkan dalam stack"
7. Method pop_2511531012() → mengambil dan menghapus elemen teratas.
   - Jika stack kosong:
     - Menampilkan "Stack Underflow"
     - Return 0
   - Jika tidak:
     - Ambil data: a[top]
     - Kurangi top: top--
8. Method peek_2511531012() → melihat elemen teratas tanpa menghapus.
   - Jika kosong:

    - Menampilkan "Stack Underflow"
    - Return 0
  - Jika tidak:
    - Mengembalikan nilai a[top]
9. Method print_2511531012() → menampilkan isi stack.
  - Loop dari top ke bawah:
    - for(int i = top; i > -1; i--)
  - Menampilkan dari atas ke bawah.

2.5 stackArrayDriver_2511531012

```java
package pekan3_2511531012;

public class stackArrayDriver_2511531012 {

    public static void main(String[] args) {
        stackArray_2511531012 s= new stackArray_2511531012();
        s.push_2511531012(10);
        s.push_2511531012(20);
        s.push_2511531012(30);
        System.out.println(s.pop_2511531012() +" dikeluarkan dari stack");
        System.out.println("Elemen teratas adalah :" + s.peek_2511531012());
        System.out.print("Elemen pada stack:");
        s.print_2511531012();

    }

}
```

Penjelasan Program:

1. stackArray_2511531012 s = new stackArray_2511531012(); → Membuat stack baru dengan kapasitas maksimal 1000 elemen.
2. Menambahkan Elemen (push)
  - s.push_2511531012(10);
  - s.push_2511531012(20);
  - s.push_2511531012(30);
3. s.pop_2511531012() → Mengambil sekaligus menghapus elemen paling atas.
   Output → 30 dikeluarkan dari stack
4. s.peek_2511531012() → Melihat elemen teratas tanpa menghapusnya.
   Output → Elemen teratas adalah :20
5. s.print_2511531012(); → Menampilkan seluruh isi stack dari atas ke bawah.
   Output → Elemen pada stack: 20 10

2.6 StackPostfix_2511531012

```java
package pekan3_2511531012;

import java.util.Scanner;

public class StackPostfix_2511531012 {
    public static int postfixEvaluate_2511531012(String expression) {
        Stack<Integer> s = new Stack<Integer>();
        Scanner input = new Scanner (expression);
        while (input.hasNext()) {
            if (input.hasNextInt()) {
                s.push(input.nextInt());
            }else {
                String operator = input.next();
                int operand2 = s.pop();
                int operand1 = s.pop();
                if (operator.equals("+")) {
                    s.push(operand1 + operand2);
                }else if (operator.equals("-")){
                    s.push(operand1 - operand2);
                }else if (operator.equals("*")) {
                    s.push(operand1 * operand2);
                }else {
                    s.push(operand1 / operand2);
                }
            }

        }
        input.close();
        return s.pop();
    }
    public static void main(String[] args) {
        System.out.println("hasil postfix= "+postfixEvaluate_2511531012("5 2 4 * + 7 -"));

    }
}
```

Penjelasan Program:

1. Method postfixEvaluate_2511531012(String expression)
2. Stack<Integer> s = new Stack<Integer>(); → Digunakan untuk menyimpan angka sementara selama proses perhitungan.
3. Scanner input = new Scanner(expression); → Digunakan untuk membaca ekspresi satu per satu (token).
4. Perulangan Membaca Token
   while (input.hasNext())
   Selama masih ada data dalam ekspresi:
   - Jika Token adalah Angka
     input.hasNextInt()
   - Maka:
     Angka dimasukkan ke stack: → s.push(input.nextInt());
   - Jika Token adalah Operator
     Operator diambil: → String operator = input.next();
     Mengambil dua angka dari stack:
     - operand2 = s.pop();
     - operand1 = s.pop();
5. Proses Operasi
   - Jika operator:
     - "+" → penjumlahan
     - "-" → pengurangan
     - "*" → perkalian
     - selain itu → pembagian /
6. Menyimpan hasil → s.push(hasil); → Hasil operasi dimasukkan kembali ke stack:
7. Mengembalikan Hasil Akhir → return s.pop(); → Nilai terakhir di stack adalah hasil akhir.

BAB III

KESIMPULAN

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa struktur data stack bekerja dengan prinsip Last In First Out (LIFO), di mana elemen terakhir yang dimasukkan akan dikeluarkan pertama. Operasi dasar seperti push, pop, dan peek dapat diimplementasikan dengan baik menggunakan berbagai cara, seperti array, ArrayList, maupun class bawaan Java.

Selain itu, stack juga dapat diterapkan dalam berbagai kasus, seperti pencarian nilai maksimum dan evaluasi ekspresi postfix. Praktikum ini membantu dalam memahami konsep stack baik secara teori maupun implementasi, serta meningkatkan kemampuan logika pemrograman dalam menyelesaikan masalah.

DAFTAR PUSTAKA

[1] I. Liem, *Draft Diktat Struktur Data*, Edisi 2008. Bandung, Indonesia: Program Studi Teknik Informatika, Institut Teknologi Bandung, 2008.

Tugas

Kelas ADT: Website_2511531012

```java
package pekan3_2511531012;

public class Website_2511531012 {
    private String judul_1012;
    private String url_1012;

    // Constructor
    public Website_2511531012(String judul, String url) {
        this.judul_1012 = judul;
        this.url_1012 = url;
    }

    // Getter
    public String getJudul_1012() {
        return judul_1012;
    }

    public String getUrl_1012() {
        return url_1012;
    }

    // Setter
    public void setJudul_1012(String judul) {
        this.judul_1012 = judul;
    }

    public void setUrl_1012(String url) {
        this.url_1012 = url;
    }

    // Menampilkan data
    public String toString() {
        return "Judul: " + judul_1012 + "\nURL: " + url_1012;
    }
}
```

Penjelasan Program:

1. Atribut (Variabel)
   - private String judul_1012; → Menyimpan judul website
   - private String url_1012; → Menyimpan alamat URL website
     →Menggunakan private agar data terenkapsulasi (tidak bisa diakses langsung).
2. Constructor → public Website_2511531012(String judul, String url)

   Mengisi nilai:
   - judul_1012
   - url_1012

3. Method Getter
   - getJudul_1012()
   - getUrl_1012()

   Fungsi: Mengambil nilai atribut (membaca data).
4. Method Setter
   - setJudul_1012(String judul)
   - setUrl_1012(String url)

   Fungsi: Mengubah nilai atribut.
5. Method toString() → Mengubah objek menjadi teks saat ditampilkan.

   Digunakan otomatis saat System.out.println(object)

Kelas Driver: Browser_2511531012

```java
package pekan3_2511531012;

import java.util.Scanner;
import java.util.Stack;

public class Browser_2511531012 {

    public static void main(String[] args) {
        Stack<Website_2511531012> history = new Stack<>();
        Scanner input = new Scanner(System.in);
        int pilihan;

        do {
            System.out.println("\n=== Browser History NIM: 2511531012 ===");
            System.out.println("1. Kunjungi Website (Push)");
            System.out.println("2. Tombol Back (Pop)");
            System.out.println("3. Lihat Halaman Aktif (Peek)");
            System.out.println("4. Cek Status History");
            System.out.println("5. Keluar");
            System.out.print("Pilihan: ");
            pilihan = input.nextInt();
            input.nextLine(); // buffer

            switch (pilihan) {

                case 1:
                    // PUSH
                    System.out.print("Masukkan Judul: ");
                    String judul = input.nextLine();

                    System.out.print("Masukkan URL: ");
                    String url = input.nextLine();

                    Website_2511531012 web = new Website_2511531012(judul, url);
                    history.push(web);
```

```java

                    System.out.println("Berhasil mengunjungi halaman!");
                    break;

                case 2:
                    // POP
                    if (!history.isEmpty()) {
                        Website_2511531012 removed = history.pop();
                        System.out.println("Kembali dari halaman:");
                        System.out.println(removed);
                    } else {
                        System.out.println("History kosong! Tidak bisa kembali.");
                    }
                    break;

                case 3:
                    // PEEK
                    if (!history.isEmpty()) {
                        System.out.println("Halaman aktif:");
                        System.out.println(history.peek());
                    } else {
                        System.out.println("Tidak ada halaman yang dibuka.");
                    }
                    break;

                case 4:
                    // CEK STATUS
                    if (history.isEmpty()) {
                        System.out.println("History kosong.");
                    } else {
                        System.out.println("Jumlah history: " + history.size());
                    }
                    break;

                case 5:
                    System.out.println("Keluar dari program...");
                    break;

                default:
                    System.out.println("Pilihan tidak valid!");
            }

        } while (pilihan != 5);

        input.close();
    }
}
```

Penjelasan Program:

1. Deklarasi Variabel
    - Stack<Website_2511531012> history = new Stack<>(); → Stack untuk menyimpan objek website, mengikuti konsep LIFO.
    - Scanner input = new Scanner(System.in); → Digunakan untuk menerima input dari user.
    - int pilihan; → Menyimpan pilihan menu user.
2. Perulangan Program
    - Menggunakan: do { ... } while (pilihan != 5);

- Program akan terus berjalan sampai user memilih keluar (5).

3. Menu Program

   Program menampilkan menu:

   1) Push (Kunjungi Website)
   2) Pop (Back)
   3) Peek (Halaman Aktif)
   4) Cek Status
   5) Keluar

4. Penjelasan Tiap Menu

   Push (Kunjungi Website)

   - User memasukkan:
     - Judul
     - URL
   - Dibuat objek: → Website_2511531012 web
   - Disimpan ke stack: → history.push(web)
   - Hasil:
     - Website masuk ke atas stack
     - Menjadi halaman aktif

   Pop (Tombol Back)

   - Mengecek: → history.isEmpty()
   - Jika tidak kosong:
     - Menghapus elemen teratas: → history.pop()
     - Menampilkan halaman yang ditutup
   - Jika kosong:
     - Menampilkan pesan: → "History kosong"

   Peek (Halaman Aktif) → Mengecek apakah stack kosong

   - Jika tidak kosong:
     - Menampilkan: → history.peek()
   - Jika kosong:
     - Menampilkan pesan: → "Tidak ada halaman"

     Tidak menghapus data

   Cek Status History

   - Jika kosong:

- Menampilkan "History kosong"

- Jika tidak:
  - Menampilkan jumlah data: → history.size()

Keluar

- Program berhenti
- Scanner ditutup

5. Alur Singkat Program
   1) User memilih menu
   2) Program menjalankan perintah sesuai pilihan
   3) Data disimpan dalam stack
   4) Program berulang sampai user keluar

Ouput

```
=== Browser History NIM: 2511531012 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. Keluar
Pilihan: 1
Masukkan Judul: SSO UNAND
Masukkan URL: https://sso.unand.ac.id/login
Berhasil mengunjungi halaman!

=== Browser History NIM: 2511531012 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. Keluar
Pilihan:
```